

Credential Stuffing

Your Password Spraying guide's closing cheat sheet drew one axis of comparison against offline cracking — this guide is the other missing comparison, since credential stuffing and password spraying are constantly confused despite being genuinely different techniques that just happen to both be **online attacks against live authentication endpoints**. Worth locking in the distinction immediately: password spraying tries **one password across many usernames**, betting on password *weakness* (a small number of common passwords covering some fraction of any population); credential stuffing tries **known, real username:password PAIRS** — sourced from actual prior breaches — against a *different* target service, betting on password *reuse* (the same human using the same credential across multiple unrelated sites). Same broad category of attack, completely different exploited weakness. **Part 1 covers the core mechanics** — why reuse rates make this viable at all, and where the credential pairs actually come from — **Part 2 covers practical execution and tooling** — the specialized automation this technique depends on at the scale it operates at, and how it evades detection differently than spraying does.

1. The Exploited Weakness — Reuse, Not Weakness — The Foundation

Every technique earlier in this series exploited some variant of a **weak** password — easily guessed, short, low-entropy, or covered by a common wordlist. Credential stuffing doesn't require the password to be weak at all:

```
A user's password for Site A: correcthorsebatterystaple2026!  
→ genuinely strong, would resist JtR/Hashcat/spraying entirely
```

```
Same user reuses the IDENTICAL password for Site B, Site C, Site D  
→ if Site A is breached and that password (or its hash, cracked  
offline per your earlier guides) becomes known, it is now a  
VALID, WORKING credential for Sites B, C, and D too - password  
STRENGTH was never the relevant variable at all
```

The single fact that makes credential stuffing viable at internet scale — password reuse across unrelated services is extremely common human behavior, independently of how strong any individual password is, meaning a breach at ONE service routinely compromises accounts at services that were

never themselves breached or even vulnerable. This is why credential stuffing is sometimes described as attacking the *user's behavior* rather than attacking any specific target's security posture — the target service can have a flawless password policy, proper salted Argon2id storage (per your Hash Types guide), universal MFA even, and still see successful account takeovers driven entirely by a breach that happened somewhere else, years earlier, against a completely unrelated company.

2. Where the Credential Pairs Actually Come From

- Publicly circulated breach compilations ("Collection #1" and similar aggregated dumps - billions of email:password pairs compiled from many individual breaches over years)
- Freshly cracked credentials from your own JtR/Hashcat pipeline (Section 2's connection back to this series' earlier guides is direct - offline cracking's OUTPUT is exactly what feeds a stuffing list's INPUT)
- Combolists traded/sold on dedicated marketplaces and forums, often pre-filtered/sorted by target site relevance, geography, or "freshness" (how recently the underlying breach occurred, since reuse rates and password-rotation likelihood both decay over time)
- Malware-harvested credentials (infostealer logs) - increasingly the dominant modern source, since these capture credentials DIRECTLY at entry time (browser autofill, saved passwords) rather than requiring the attacker to breach and crack a separate service first

PART 1: Core Mechanics & Reuse Dynamics

3. Why This Is a Fundamentally Different Bet Than Spraying

Password spraying's bet: "SOME fraction of this specific target's user population chose a common, guessable password" - succeeds or fails based on THIS target's password strength distribution

Credential stuffing's bet: "SOME fraction of these known, real credential pairs are REUSED by their owner on THIS target" - succeeds or fails based on REUSE RATE, a property of human behavior largely independent of this target's own security posture

Published reuse-rate research consistently finds a MEANINGFUL double-digit percentage of users reuse passwords across multiple services - the exact figure varies by study/population, but the

directional finding is stable and large enough that stuffing a sufficiently large combolist against a sufficiently large target user base reliably yields SOME successful account takeovers, even against a target with a genuinely strong password policy

This is why credential stuffing remains effective against organizations that have already done everything "right" on Section 15 of your Hash Types guide and Section 13 of your Password Spraying guide — those defenses protect against weak passwords and against this-target-specific guessing, not against a credential compromised somewhere else entirely.

4. Success Rate Math — Why Scale Compensates for a Low Per-Attempt Rate

Even a LOW per-credential success rate (commonly cited real-world figures for large stuffing campaigns run well under 1-3%) becomes a meaningful ABSOLUTE number of compromised accounts once applied against a combolist containing millions or billions of pairs:

1% success rate x 10,000,000 credential pairs tested against a target = 100,000 potentially valid account takeovers

This is the credential-stuffing equivalent of your Password Spraying guide's Section 2 "weakest link" statistical principle - here the relevant population is CREDENTIAL PAIRS rather than TARGET USERS, but the same "you only need a small fraction to hit" logic drives viability at scale

5. Targeting Overlap — Why Certain Sectors See Disproportionate Stuffing Traffic

- Financial services, e-commerce, and streaming/subscription services see disproportionately heavy stuffing traffic, since a successful hit has DIRECT monetizable value (fraudulent purchases, resellable subscription access, financial account access)
- Corporate/enterprise login portals are targeted specifically because of Section 3's reuse dynamic combined with the high value of a single successful enterprise account compromise as an initial-access foothold - directly relevant if you're chaining this into a broader engagement alongside your Password Spraying guide's AD/cloud-identity targeting

PART 2: Practical Execution & Tooling

6. Purpose-Built Stuffing Tools — OpenBullet and Similar

Unlike Kerbrute/NetExec's spraying focus (your Password Spraying guide's Sections 7-8), credential stuffing has its own dedicated tooling ecosystem built specifically around processing enormous combolists against arbitrary web login forms at scale:

```
OpenBullet2 (and its predecessor's legacy config format) is the
most widely referenced tool in this space - built around:
- "Config" files: reusable, shareable definitions of HOW to
  interact with a specific target's login flow (request format,
  cookie/token handling, success/failure response detection)
- Combolist input: the email:password pairs to test, one per line
- Proxy list integration: rotating source IPs across attempts
  (Section 8 below)
- Multi-threaded execution: testing many credential pairs
  concurrently against the target
```

The "config" concept is worth understanding structurally even without running the tool yourself — it's effectively a small, target-specific automation script capturing exactly what a legitimate login request/response cycle looks like, so the tool can distinguish a successful login from a failed one at scale, automatically, across millions of attempts.

7. Success/Failure Detection Logic — The Core Automation Challenge

```
A stuffing run against millions of pairs needs to AUTOMATICALLY
classify each attempt without manual review - typically via:
- HTTP status code / redirect behavior (a successful login often
  redirects to a distinct authenticated-area URL)
- Response body keyword matching ("Invalid password" vs a
  dashboard/welcome page's distinct HTML)
- Session cookie presence/absence after the request completes
- Response TIMING differences (a subtle but real signal - some
  applications process valid-username-invalid-password vs
  fully-invalid-credential requests at measurably different
  speeds, an information leak worth flagging in Section 12's
  prevention notes)
```

8. Proxy Rotation and Residential IP Networks

Scale is the entire premise here, and scale from a single source IP is trivially rate-limited/blocked — meaningfully more so than the Password Spraying guide's Section 10 OPSEC concern, since stuffing volumes are typically orders

of magnitude higher (millions of attempts vs a spray's carefully-limited handful of rounds):

- Datacenter proxy lists: cheap, high-volume, but increasingly trivially detected/blocked by modern anti-automation services (datacenter IP ranges are well-cataloged and heavily scrutinized)
- Residential proxy networks: route traffic through real consumer ISP connections (sometimes via compromised IoT devices or consent-questionable "free VPN"/SDK-bundled proxy networks), making traffic appear to originate from ordinary home users rather than obviously automated infrastructure - a genuinely significant escalation in evasion difficulty for defenders compared to Section 10 of your Password Spraying guide's simpler IP-rotation concern

9. CAPTCHA and Bot-Detection Evasion

Distinct from anything in your Password Spraying guide, since spraying's low volume rarely triggers behavioral bot-detection the way stuffing's sheer scale does:

- CAPTCHA-solving services (human-solver farms or, increasingly, automated ML-based solvers) integrated directly into stuffing tool pipelines
- Browser fingerprint spoofing / headless-browser detection evasion, since many modern anti-automation platforms fingerprint TLS handshakes, canvas rendering, and JS execution environment characteristics rather than relying on rate/IP alone
- Deliberately human-like request timing/mouse-movement simulation to defeat behavioral bot-scoring systems

This entire category of evasion is a direct, escalating arms race against the anti-automation/bot-management vendor space (Section 13's prevention section is essentially the defender's side of exactly this arms race).

10. Testing / Engagement Methodology

1. Confirm authorization explicitly covers credential-stuffing testing specifically - this carries the SAME live-account risk profile as your Password Spraying guide's Section 12, but at potentially far larger volume/scale, meaningfully raising the business-disruption risk that needs explicit client sign-off
2. Source a REALISTIC combolist for the engagement - ideally credentials already known/breached for this SPECIFIC organization's user base (from prior authorized findings or

- client-provided historical breach data), not a generic public combolist, to keep the test meaningful and avoid unnecessarily testing accounts with no plausible reuse connection to the target
3. Configure detection logic (Section 7) carefully and validate it against a known test account BEFORE running at scale, to avoid false-positive "successes" wasting engagement time
 4. Respect realistic THROTTLING even in an authorized test - running at true malicious-actor scale/speed risks genuine service disruption; agree on a representative but safer volume/rate with the client in advance
 5. Report the ACTUAL reuse rate observed (Section 4's math, made concrete for this specific client) as the core finding - this is usually a far more persuasive data point for driving MFA/reuse-prevention investment than a generic industry statistic

11. Legal & Ethical Note

Identical baseline authorization requirement to every prior guide, with the SAME elevated live-account risk profile flagged in your Password Spraying guide's Section 12 - additionally, sourcing or possessing breach-derived combolists carries its own separate legal considerations in many jurisdictions independent of how they're subsequently used, and organizations increasingly restrict even security researchers from downloading/redistributing certain breach compilations. Use only data you are explicitly authorized to possess and test with, sourced through legitimate/documented channels.

12. Prevention

- Enforce MFA universally - identical to your Password Spraying guide's Section 6/13, and for the exact same reason: it converts a successful credential match from account takeover into merely a confirmed-reuse data point, and it's the single control that remains effective REGARDLESS of how strong the target's own password policy is, since Section 1 established that strength was never the exploited variable here to begin with
- Screen credentials at LOGIN time (not just at password-set time) against known-breached corpora, using a service like Have I Been Pwned's Pwned Passwords API or equivalent - directly detects Section 3's reuse condition proactively, before an attacker does
- Deploy bot-management/anti-automation tooling capable of detecting Section 8-9's evasion techniques specifically (behavioral scoring, TLS/browser fingerprinting, residential-proxy detection heuristics) rather than relying purely on simple rate-limiting, which Section 8 is specifically built to route around
- Monitor for IMPOSSIBLE TRAVEL and anomalous login patterns (successful login from a geographically implausible location shortly after a legitimate one) as a POST-compromise detection

layer, since Section 4's math guarantees some volume of successful stuffing will occur against any sufficiently large user base regardless of preventive controls

- Avoid leaking Section 7's detection-relevant timing/response differences between "valid username, wrong password" and "invalid username" responses - a subtle but real information leak that materially assists automated success/failure classification

13. Practical Lab Example

Building a small local lab web app with a deliberately reused test-account password, then running OpenBullet2 (or a simple custom script replicating its core request/response classification logic) against a combolist that includes that reused pair among many invalid ones, is the clearest way to see Section 4's scale math proven directly - most of the list fails, exactly the expected handful succeed.

Suggested practice order:

1. Set up a local test login form and a combolist where you KNOW the exact reuse rate in advance (e.g. 1 valid pair out of 1,000)
2. Build or configure a simple automated success/failure classifier (Section 7) and run it against the full list, confirming it correctly identifies the single valid pair without manual review
3. Compare this exercise directly against your Password Spraying guide's Section 14 lab - same live-authentication-endpoint category of attack, deliberately different exploited weakness (reuse here vs weak-password-guessing there), same detection principle (aggregate anomalous login pattern) defending against both

Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

Framework	Mapping
CWE	CWE-307 (Improper Restriction of Excessive Authentication Attempts - same CWE as your Password Spraying guide, since both ultimately abuse insufficient rate/volume restriction on live authentication attempts, just via a different credential-selection strategy), CWE-521 is notably LESS relevant here than in any earlier guide, since Section 1 established password strength isn't the exploited variable
OWASP Top 10:2025	A07:2025 Identification and Authentication Failures - identical category to your Password Spraying guide; OWASP's own dedicated Credential Stuffing Cheat Sheet is a direct, authoritative extension of this section

Framework	Mapping
CVSS	Typically High for a successful individual account takeover absent MFA (identical profile to spraying), but the AGGREGATE business risk of a large-scale campaign is usually assessed separately from any single account's CVSS score, given Section 4's volume-driven impact model
Cyber Kill Chain	Reconnaissance (Section 2's combolist sourcing, which notably happens largely OUTSIDE the target's own environment, unlike your Password Spraying guide's target-specific username enumeration) → Delivery/Exploitation (the stuffing attempts against the live endpoint)
MITRE ATT&CK	T1110.004 (Brute Force: Credential Stuffing - its own dedicated sub-technique, sibling to T1110.003's Password Spraying and T1110.002's offline cracking covered throughout this series), T1589.001 (Gather Victim Identity Information: Credentials, covering Section 2's sourcing phase)

Quick Reference Cheat Sheet

CREDENTIAL STUFFING vs PASSWORD SPRAYING - the comparison this entire guide is built around:

	STUFFING	SPRAYING
Input:	known username:password PAIRS from prior breaches	usernames only, guessed passwords
Exploited weakness:	password REUSE across sites	password WEAKNESS at THIS target
Volume:	millions of pairs, typically (5-20) passwords per round	small handful
Detection evasion:	proxy rotation, CAPTCHA-solving, bot-fingerprint spoofing	lockout-window timing, IP distribution
Primary defense:	MFA + breach-corpus screening at login time	MFA + smart lockout + weak-pw screening
Shared defense:	MFA universally (defeats BOTH regardless of which credential-selection strategy is used)	

WORKFLOW:

1. Source/build a combolist (breach compilations, own cracking output, infostealer logs)
2. Configure a target-specific success/failure classifier (OpenBullet-style config)
3. Run at scale through rotated proxies + CAPTCHA evasion
4. Harvest successful pairs; assess value (financial, corporate foothold, resale)

Root lesson: this technique attacks REUSE, a property of human behavior across UNRELATED services, meaning a target's own security posture - however strong - cannot fully defend against it alone; MFA is the one control that remains effective regardless of where the compromised credential originally leaked from.
